

MODULE NAME: FUNDAMENTAL OF DATA STRUCTURE

CLASS: OD24ERE.

NAME: SHILLA MTERI IZINA.

REG:NUMBER: 24032403947

1. To rank the following complexities in term of their rate of growth - Start with slowest to fastest

$$O(n), O(\log n), O(n^5), O(n \log n), O(1).$$

Here before arranging lets know each one (are in order ready):-

$O(1)$ - This is constant time so does not grow with input size

$O(\log n)$ - logarithmic growth, very slow increase.

$O(n)$ - Linear growth

$O(n \log n)$ - Slightly faster than linear common in efficient - sorting algorithms.

$O(n^5)$ - polynomial growth, this is much faster than each other.

So the arrangement from slowest to fastest will be :-

$$\underline{O(1) < O(\log n) < O(n) < O(n \log n) < O(n^5)}$$

2. Finding Big O for each function.

(a) $4n^2 + 20n + 9$

The leading term = n^2

lowest terms ($20n, 9$) become negligible ~~by term~~ for large n .

$$= O(n^2).$$

$\therefore$ answer

$$= O(n^2)$$

(b) $25n^3 + 15$

Leading term = n^3

The constant 15 will be negligible.

$$= O(n^3)$$

ans.

(c) $n^3 + 5 \log n + 6$

The leading term = n^3

- The logarithm and constant term are negligible compared to cubic growth.

So will be

$$O(n^3)$$

ans.

(d) $10^5 + 10^5 n + 10^5 n^2$

The leading term = n^2

Constant and linear terms are smaller.

So the answer will be

$$= O(n^2)$$

ans.

(e) $72n(n + 3n^2)$

$$\text{This will be } = 72n^2 + 216n^3$$

the leading term = n^3

$$= O(n^3)$$

ans.

$$(f) 2^8 + 32^4 n - 5$$

here:

$$2^8 = 256$$

$$32^4 = 1048576$$

So will be:

$$256 + 1048576n - 5$$

The leading term here = n .

$$= O(n)$$

ans.

Q3 To design an algorithm that runs in worst-case $O(n)$ time to add all the numbers from 1 to a given n (including n).

Ans:

This can be like:

```
Sum := 0
for i := 1 to n do
    Sum := Sum + i
end for

Print sum
```

So the loop will run n times and each iteration does constant work.
and the complexity = $O(n)$.

Q4 (a) The complexity in terms of variable number.

```
X = 4
for i := 0 to number do:
    X := X + 1
end for
```

So the loop runs $number$ times and each operation is constant time.

Complexity = $O(n)$.

4. (b). The complexity

Soln.

Given: ~~Chose~~

Choice := get choice ()

```
if choice == 1:
    print "Ready"
```

```

else if choice = 2 then;
    for i:= 0 to cycles do;
        print i
    end for
end if
end while
end

```

And if.

from there.

The complexity = $O(n)$
ans.

4. (c) Answer .

The complexity of calculating function:

for $i := 1$ to iter do:

here will run iter times.

So complexity = $O(\text{iter})$.

calculate()

Calculate ()

for $i := 1$ to iter do :

Val := Val + increase(Val, iter)

The outer loop runs

iter.

inside it calls

increase(Val, iter) = $O(\text{iter})$

So

iter x iter

Therefore will be iter x iter = $O(\text{iter}^2)$

ans = $O(\text{iter}^2)$.

4. (d). The complexity of the calculate method.

Soln.

will be.

increase(Val, i)

here are not iter anymore, it changes with i.

will become:

$1 + 2 + 3 + \dots + \text{iter}$

The natural number formula:-

$\frac{\text{iter}(\text{iter} + 1)}{2}$

So the ans will be $O(\text{iter}^2)$.

5. (i) for some functions $f(n)$ and $g(n)$, if $f(n)$ is $O(g(n))$, then for some constant C and k for all $n \geq k$, $f(n) \leq Cg(n)$.

ans. ~~$f(n) \geq Cg(n)$~~

$$f(n) \leq Cg(n)$$

- (ii) for some functions $f(n)$ and $g(n)$, if $f(n)$ is $\Omega(g(n))$, then for some constants C and k , for all $n \geq k$, $f(n) \geq Cg(n)$.

ans.
 $f(n) \geq Cg(n)$

6. Show that $4n + 12$ is $O(n)$. Hint find two suitable constants for C and k .

Soln.

To show that $4n + 12$ is $O(n)$.

$$= 4n + 12 \leq Cn$$

$$C = 5$$

$$4n + 12 \leq 5n$$

$$12 \leq n$$

$$k = 12$$

Therefore

$$n \geq 12,$$

$$= 4n + 12 \leq 5n$$

hence

$$\underline{\underline{4n + 12 = O(n)}}$$

Q7 To show that $5n^2 - 16$ is $O(n^2)$.

Soln.

$$5n^2 - 16 \leq Cn^2$$

$$C = 5$$

$$5n^2 - 16 \leq 5n^2$$

$$K=1$$

$$\therefore 5n^2 - 16 = O(n^2)$$

hence shown